

Java OOP Exam — Questions & Correct Answers

University of Westminster | IIT Sri Lanka

Q1 — Threading — t1.run() vs t1.start()

The following code was written to display interleaved messages from Thread T1 and T2. However, it only displays 'Thread T1' forever. Which change fixes this?

```
class T1 extends Thread { public void run() { while(true) { System.out.println("Thread T1"); } } }  
class T2 extends Thread { public void run() { while(true) { System.out.println("Thread T2"); } } }  
// Main: t1.run(); // line 20 | t2.start(); // line 21
```

Correct Answer: D — Replace line 20 with: t1.start();

Student Selected: D

✓ **Student answer:** CORRECT

Explanation: t1.run() executes run() on the main thread directly, blocking forever inside the infinite while loop — t2 never gets a chance to start. Calling t1.start() spawns a new thread, allowing both threads to run concurrently.

Q2 — OOP — NOT a characteristic

Which of the following is NOT a characteristic of object-oriented programming?

Correct Answer: A — Instructional decomposition

Student Selected: A

✓ **Student answer:** CORRECT

Explanation: The four pillars of OOP are: Polymorphism, Abstraction, Encapsulation, and Inheritance. 'Instructional decomposition' is a concept from procedural/structured programming, not OOP.

Q3 — PART B — Programming (40%) — Fruit Machine Simulation

Implement Java classes to simulate a casino fruit machine scenario: - Casino has 10 Fruit Machines; player inserts 1 coin (£1) per game - Machine displays 3 random fruits from a pool of 21 fruits - 2 same fruits → player credited £20 | 3 same fruits → player credited £100 - Credit > 300: 10% chance of 2-match; Credit > 500: 2% chance of 3-match - Credit < 300: always 3 unique fruits - Player: name, address, city, balance, favourite casino - Casino: location, max 100 customers - Also implement a Simulation class: 1 player plays 100 games on 1 machine - Only 1–2 getters/setters required

Correct Answer: Open-ended written answer (handwritten on paper, scanned as PDF)

Student Selected: N/A

? **Student answer:** N/A or unreadable

Explanation: Required classes: Player (name, address, city, balance, favouriteCasino), Casino (city, customers[]), FruitMachine (coins, playGame()), Simulation (runs 100 games). Key logic: Random fruit selection with probability gates based on coin credit.

Q4 — Constructors — how many can a class have?

How many constructors can a Java class have?

Correct Answer: B — Any number

Student Selected: B

✓ **Student answer: CORRECT**

Explanation: Java supports constructor overloading. A class can have any number of constructors provided each has a unique signature (different parameter types/count). If no constructor is defined, the compiler provides a default no-arg constructor.

Q5 — equals() — Object vs String override

What is the output?

```
Q7 q1 = new Q7(100); Q7 q2 = new Q7(100);
if (q1.equals(q2)) println(true); else println(false); // Q7 does NOT override equals()
String str1 = new String("123"); String str2 = new String("123");
if (str1.equals(str2)) println(true); else println(false); // String DOES override equals()
```

Correct Answer: B — false / true

Student Selected: B

✓ **Student answer: CORRECT**

Explanation: Q7 does not override equals(), so it uses Object.equals() which compares references — q1 and q2 are different objects → false. String overrides equals() to compare content character-by-character → str1 and str2 have the same content '123' → true.

Q6 — Polymorphism — runtime dispatch

What is the output?

```
interface F { void foo(); }
class Y implements F { public void foo() { print("Y "); } }
class Q6 extends Y { public void foo() { print("Q6 "); } }
F f1 = new Q6(); f1.foo(); // runtime type is Q6
F f2 = new Y(); f2.foo(); // runtime type is Y
Y y1 = new Q6(); y1.foo(); // runtime type is Q6
```

Correct Answer: B — Q6 Y Q6

Student Selected: B

✓ **Student answer: CORRECT**

Explanation: Java uses dynamic dispatch. Even though f1 is declared as type F and y1 as type Y, the actual runtime type determines which foo() executes. f1 and y1 both hold Q6 instances → Q6.foo() runs. f2 holds a Y instance → Y.foo() runs.

Q7 — Static fields — shared across instances

What is the output?

```
class M { int x; static int y = 1;
M(int x) { this.x = x; ++y; } }
M m1 = new M(10); // m1.x=10, y→2
M m2 = new M(20); // m2.x=20, y→3
m2.y++; // y→4 (static: shared)
m1.x++; // m1.x→11
println(m1.x + " " + m2.x + " " + m1.y + " " + m2.y)
```

Correct Answer: A — 11 20 4 4

Student Selected: C — 11 20 2 3

✗ **Student answer:** WRONG

Explanation: y is static, shared by all instances. After constructing m1 and m2, y=3. m2.y++ increments the shared y to 4. m1.y and m2.y both read the same static field → 4. Student error: likely forgot that ++y fires twice in the constructor (once per new M()), and that m2.y++ also increments the shared static.

Q8 — super keyword — correct usage

Which of the following statements about the super keyword is true?

Correct Answer: C — super can call a constructor, a method, or access a field of the parent class

Student Selected: C

✓ **Student answer:** CORRECT

Explanation: super() calls the parent constructor; super.method() calls a parent method; super.field accesses a parent field. All three uses are valid. Option A is wrong (super is limited to the parent, not any class). Option E is wrong (super() CAN call a parent constructor).

Q9 — String immutability & reference semantics

What is the output?

```
String x = new String("a dog ");
String y = "a cat ";
String z = x + y; // z = "a dog a cat "
x = z; // x now points to "a dog a cat "
x = y; // x now points to "a cat "
println(z + "having a nice day")
```

Correct Answer: B — a dog a cat having a nice day

Student Selected: B

✓ **Student answer:** CORRECT

Explanation: z was assigned x+y = 'a dog a cat '. Strings are immutable — reassigning x does not affect z. z retains its value throughout, so z + 'having a nice day' = 'a dog a cat having a nice day'.

Q10 — Pass by value vs pass by reference

What is the output?

```
void modify(int i) { i++; } // primitive: copy passed, no effect on caller
void modify(MyNumber m) { m.i++; } // object: field mutated via reference
int i = 4; q5.modify(i); // i still 4
MyNumber m = new MyNumber(2); q5.modify(m); // m.i → 3
println(i + " " + m.i)
```

Correct Answer: B — 4 3

Student Selected: B

✓ **Student answer:** CORRECT

Explanation: Java passes primitives by value — modify(int i) increments a local copy, the caller's i stays 4. Objects pass a copy of the reference — modify(MyNumber m) mutates the actual object's field, so m.i goes from 2 to 3.

Q11 — Constructor chaining — implicit super()

What is the output?

```
class Parent { Parent() { print("Parent "); } }  
class Child extends Parent { Child() { print("Child "); } } // implicit super() → Parent first  
class GrandChild extends Child {  
    GrandChild() { print("Grandchild "); } // implicit super() → Child → Parent  
    GrandChild(int x) { } // implicit super() → Child → Parent only  
}  
new GrandChild(); // → Parent Child Grandchild  
new GrandChild(5); // → Parent Child  
println();
```

Correct Answer: D — Parent Child Grandchild Parent Child

Student Selected: D

✓ **Student answer: CORRECT**

Explanation: Every constructor implicitly calls super() as its first action unless explicitly stated otherwise. new GrandChild() chains: Parent() → Child() → GrandChild(). new GrandChild(5) has an empty body but still chains: Parent() → Child() → GrandChild(int x) [empty]. println() adds a newline.

Q12 — Access modifiers across packages

Which lines produce a compiler error? (3 files: AccessQ13.java in f1, A.java in f1, B.java in p2)

```
// A.java (package f1): int i [package-private]; protected int j; private int k;  
// AccessQ13 (same package f1): a1.i=6 [line 6] OK | a1.j=7 [line 7] OK | a1.k=8 [line 8] ERROR  
// B extends A (package p2): i=5 [line 15] ERROR | j=10 [line 16] OK | k=20 [line 17] ERROR
```

Correct Answer: C — lines 8, 15, and 17

Student Selected: B — lines 8, 16, and 17

✗ **Student answer: WRONG**

Explanation: Line 8: k is private → inaccessible outside A, even in same package → ERROR. Line 15: i is package-private (default) → inaccessible from a different package (p2), even via inheritance → ERROR. Line 16: j is protected → accessible in subclasses across packages → OK. Line 17: k is private → inaccessible in subclass → ERROR. Student error: confused protected (j, line 16) as inaccessible and missed that package-private (i, line 15) is inaccessible cross-package.

Q13 — Object reference aliasing

What is the output? (class D: private int x, private int y=5; no explicit constructor)

```
D a1=new D(); D a2=new D(); D a3=new D(); // a1,a2,a3: x=0,y=5  
a1.x = a3.y; // a1.x = 5  
a3 = a1; // a3 → a1's object  
a1 = a3; // no change (already same)  
a2.x = 12; // a2.x = 12  
a3.x = a3.x+2; // shared obj (a1/a3): x = 7  
a1.x = 30; // shared obj: x = 30
```

```
a1.y = 21; // shared obj: y = 21
a2.y = 22; // a2.y = 22
println(a1.x+a2.x+a3.x+a1.y+a2.y) // → 30 12 30 21 22
```

Correct Answer: 30 12 30 21 22 (options partially unreadable in source — verify against mark scheme)

Student Selected: Unknown (continuation image unreadable)

? **Student answer:** N/A or unreadable

Explanation: After a3=a1, a1 and a3 alias the same object. All mutations through a1 or a3 affect the same object. a2 remains independent. Key trace: shared object ends at x=30, y=21; a2 ends at x=12, y=22.

Q14–Q15 — Questions unreadable

Source images for Q14 and Q15 were completely black — content unavailable.

Correct Answer: Unknown — images not readable

Student Selected: Unknown

? **Student answer:** N/A or unreadable

Explanation: Re-upload clear screenshots to review these questions.

Q16 — String == vs equals() with literals vs new String()

What is the output?

```
String string = "STRING"; // literal → interned in string pool
if ("STRING".equals("STRING")) // content compare → true → print "Equal "
if (string == "STRING") // reference compare: both point to interned literal → true →
print "Equal "
```

Correct Answer: D — Equal Equal

Student Selected: C — Equal Not Equal

✗ **Student answer:** WRONG

Explanation: First check: 'STRING'.equals('STRING') compares content → true → 'Equal '. Second check: string was assigned from a literal, so it is interned. The literal 'STRING' on the right is also interned. Both references point to the same object in the string pool → == returns true → 'Equal '. Student error: assumed == always fails for String objects, but this only holds when String is created with 'new String(...)'. Literal assignment uses the interned pool, making == work.

Summary — Correct vs Incorrect

Status	Questions	Count
✓ Correct	Q1, Q2, Q4, Q5, Q6, Q8, Q9, Q10, Q11	9
✗ Wrong	Q7, Q12, Q16	3
? N/A	Q3, Q13, Q14–Q15	3